

Advanced Machine Learning

DATA 642

Lecture 13 — Back propagation and challenges with learning

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Multi-Layer Perceptron and Backpropagation

The idea is based on Gradient Descent using an efficient technique for computing the gradients automatically: in just two passes through the network (one forward, one backward), the backpropagation algorithm is able to compute the gradient of the network's error with regards to every single model parameter. In simple words it can find out how each connection weight and each bias term should be tweaked in order to reduce the error. Once it has these gradients, it just performs a regular Gradient Descent step, and the whole process is repeated until the network converges to the solution.

The algorithm

- It handles one mini-batch at a time (for example containing 32 instances each), and it goes through the full training set multiple times. Each pass is called an epoch.
- Each mini-batch is passed to the network's input layer, which just sends it to the first hidden layer. The algorithm then computes the output of all the neurons in this layer (for every instance in the mini-batch). The result is passed on to the next layer, its output is computed and passed to the next layer, and so on until we get the output of the last layer, the output layer. This is the forward pass: it is exactly like making predictions, except all intermediate results are preserved since they are needed for the backward pass.
- Next, the algorithm measures the network's output error (i.e., it uses a loss function that compares the desired output and the actual output of the network, and returns some measure of the error).

The algorithm (cont.)

- Then it computes how much each output connection contributed to the error. This is done analytically by simply applying the chain rule (perhaps the most fundamental rule in calculus), which makes this step fast and precise.
- The algorithm then measures how much of these error contributions came from each connection in the layer below, again using the chain rule and so on until the algorithm reaches the input layer. This reverse pass efficiently measures the error gradient across all the connection weights in the network by propagating the error gradient backward through the network (hence the name of the algorithm).
- Finally, the algorithm performs a Gradient Descent step to tweak all the connection weights in the network, using the error gradients it just computed.

Pseudo-algorithm for forward propagation

Input:

- Input features x
- Weight matrices W
- Bias vectors b
- Activation functions σ

Procedure ForwardPropagation(x, W, b, σ): # Initialize input layer with input features $a_{\text{prev}} = x$

```
# Iterate over each layer in the network
for each layer l:
    # Compute the pre-activation value for the current layer
     $z = W[l] * a_{\text{prev}} + b[l]$ 

    # Apply the activation function to the pre-activation value to get the activation value
     $a = \sigma(z)$ 

    # Set the activation value of the current layer as the input for the next layer
     $a_{\text{prev}} = a$ 

# Output the final activation value (prediction)
return a
```


Pseudo-algorithm for the backward propagation step

Procedure BackwardPropagation($x, y, W, b, \sigma, \sigma'$): # Initialize gradients dW = empty list of gradients with same shape as W db = empty list of gradients with same shape as b

```
# Perform forward propagation to compute activations
a = ForwardPropagation(x, W, b,  $\sigma$ )

# Compute initial error gradient
delta =  $\partial L / \partial a * \sigma'(z)$ , where  $L$  is the loss function

# Iterate backward through the layers to compute gradients
for each layer  $l$  from last to first:
    # Compute gradients of parameters
     $dW[l] = \text{delta} * a[l-1].T$ 
     $db[l] = \text{sum}(\text{delta}, \text{axis}=0)$ 

    # Compute gradient with respect to the previous layer's activation
     $\text{delta} = (W[l].T * \text{delta}) * \sigma'(z)$ , where  $*$  denotes matrix multiplication

# Compute gradient of the first layer
 $dW[0] = \text{delta} * x.T$ 
 $db[0] = \text{sum}(\text{delta}, \text{axis}=0)$ 

# Output gradients of parameters
return  $dW, db$ 
```

Automatic Differentiation for Backpropagation

A popular method for calculating derivatives and applying the chain rule in backpropagation is known as **automatic differentiation** (AD). Automatic differentiation is a computational technique that computes derivatives of a function efficiently and accurately, typically by decomposing it into a sequence of elementary arithmetic operations and elementary functions for which derivatives are known.

There are two main approaches to automatic differentiation:

1. **Forward Mode:** In forward mode automatic differentiation, derivatives are computed by applying the chain rule from the input variables towards the output. This method is efficient for functions with few inputs and many outputs.
2. **Reverse Mode:** In reverse mode automatic differentiation, derivatives are computed by applying the chain rule from the output variables towards the input. This method is particularly efficient for functions with many inputs and few outputs, which is common in neural network backpropagation.

Reverse mode automatic differentiation is extensively used in training neural networks because it allows gradients to be efficiently computed with respect to a large number of parameters, which is essential for optimizing the parameters through techniques like stochastic gradient descent (SGD) or its variants.

Challenges in Neural Network Optimization

Local Minima:

- **Description:** Local minima are points in the parameter space where the loss function reaches a minimum value, but it's not the global minimum. Optimization algorithms may get stuck in local minima, failing to find the globally optimal solution.

Plateaus, Saddle Points, and Other Flat Regions:

- **Description:** Plateaus, saddle points, and other flat regions are areas of the parameter space where the gradient of the loss function is close to zero. Optimization algorithms may struggle to make progress, leading to slow convergence.
- **Example:** In high-dimensional parameter spaces typical of neural networks, there can be vast regions where the gradient is close to zero. This often occurs near the optimum, making it challenging for optimization algorithms to distinguish between flat regions and the true minimum.

Cliffs and Exploding Gradients:

- **Description:** Cliffs are steep regions in the loss landscape where small changes in parameters lead to large changes in the loss function. This can cause gradients to explode, making optimization unstable.
- **Example:** In deep neural networks with many layers, gradients can accumulate and become very large or very small, especially during backpropagation. This phenomenon is often observed in recurrent neural networks (RNNs) and deep convolutional neural networks (CNNs).

Long-term Dependencies:

- **Description:** Long-term dependencies refer to situations where the prediction at a certain time step depends on inputs or events that occurred far in the past. In recurrent neural networks (RNNs), capturing such dependencies can be challenging due to vanishing or exploding gradients.
- **Example:** In natural language processing tasks such as language modeling or machine translation, understanding the context of a word often requires considering words from distant parts of the text. RNNs may struggle to learn such dependencies effectively.

Inexact Gradients:

- **Description:** Inexact gradients arise when errors occur during the computation of gradients. This can result from numerical instability, approximation errors, or limitations in the gradient estimation process.
- **Example:** When using numerical differentiation to approximate gradients, small numerical errors can accumulate, leading to inaccurate gradient estimates. Additionally, in distributed training settings, communication delays or hardware failures can introduce errors in gradient updates.

Poor Correspondence Between Local and Global Structure:

- **Description:** This challenge arises when the local structure of the optimization landscape does not accurately reflect the global structure. Optimization algorithms may be misled by deceptive local optima or regions that seem promising locally but do not lead to the global optimum.
- **Example:** In non-convex optimization problems, the loss landscape may contain multiple local minima, saddle points, and flat regions. Navigating through such landscapes can be challenging for optimization algorithms, especially if the local optima are deceptive and trap the optimization process.